UNIVERSITY OF TEXAS ARLINGTON

INFORMATION TECHNOLOGY LABORATORY @ UTA-CSE

# CSE 3330/5330-008 Project Presentation

## Team #3

**Rohita Konjeti, Haley Johnson, Whiton Nguyen**

# Project Overview

## TV Series Review and Tracking Management System

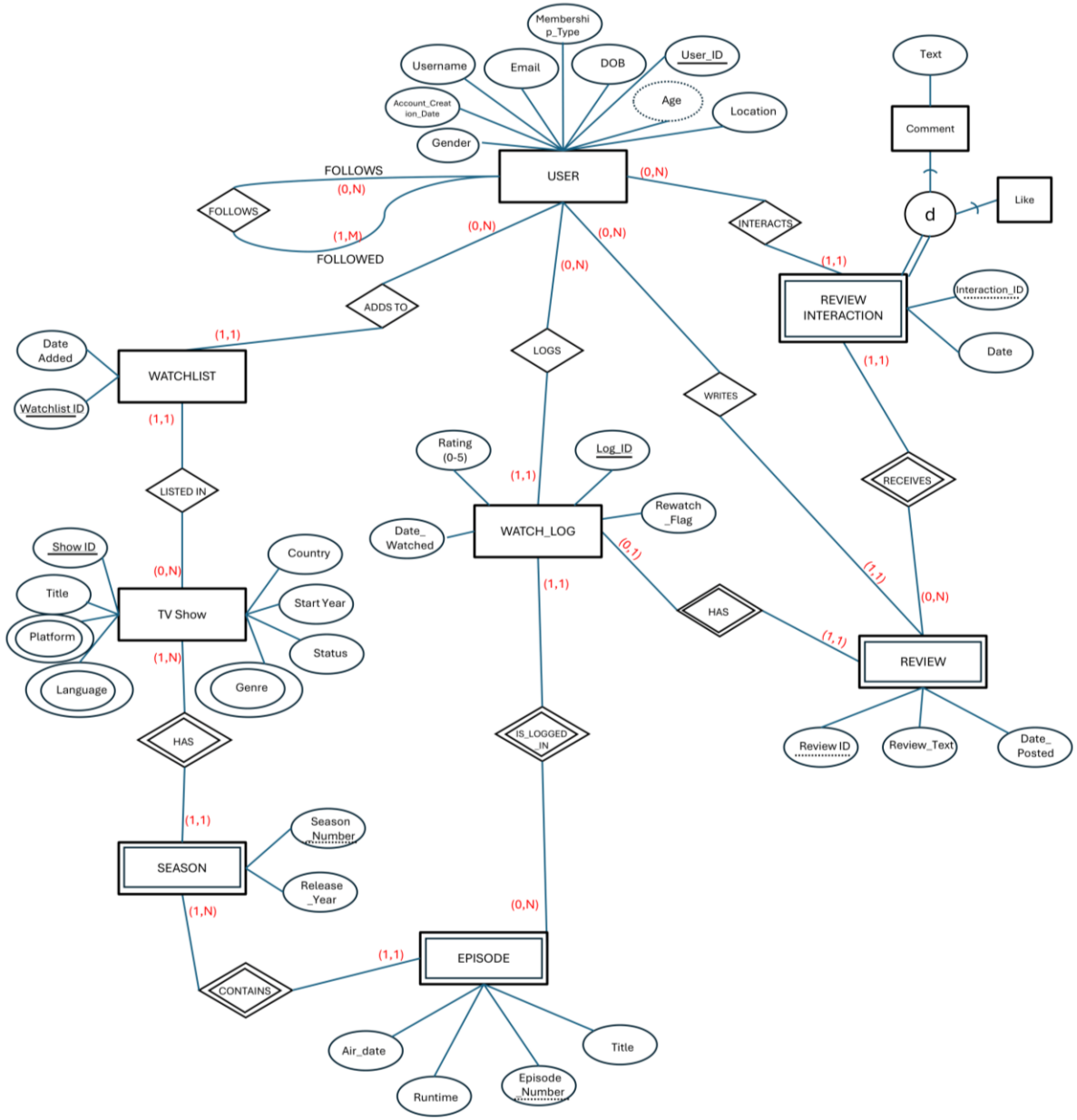
Domain: Entertainment and Streaming

We developed a database system for an all-inclusive TV Show management system which supports progress tracking within seasons and episodes. Furthermore, our system allows users to rate and review content, manage watchlists, and interact through likes, comments and follows. This system captures robust user data across multiple shows over time.

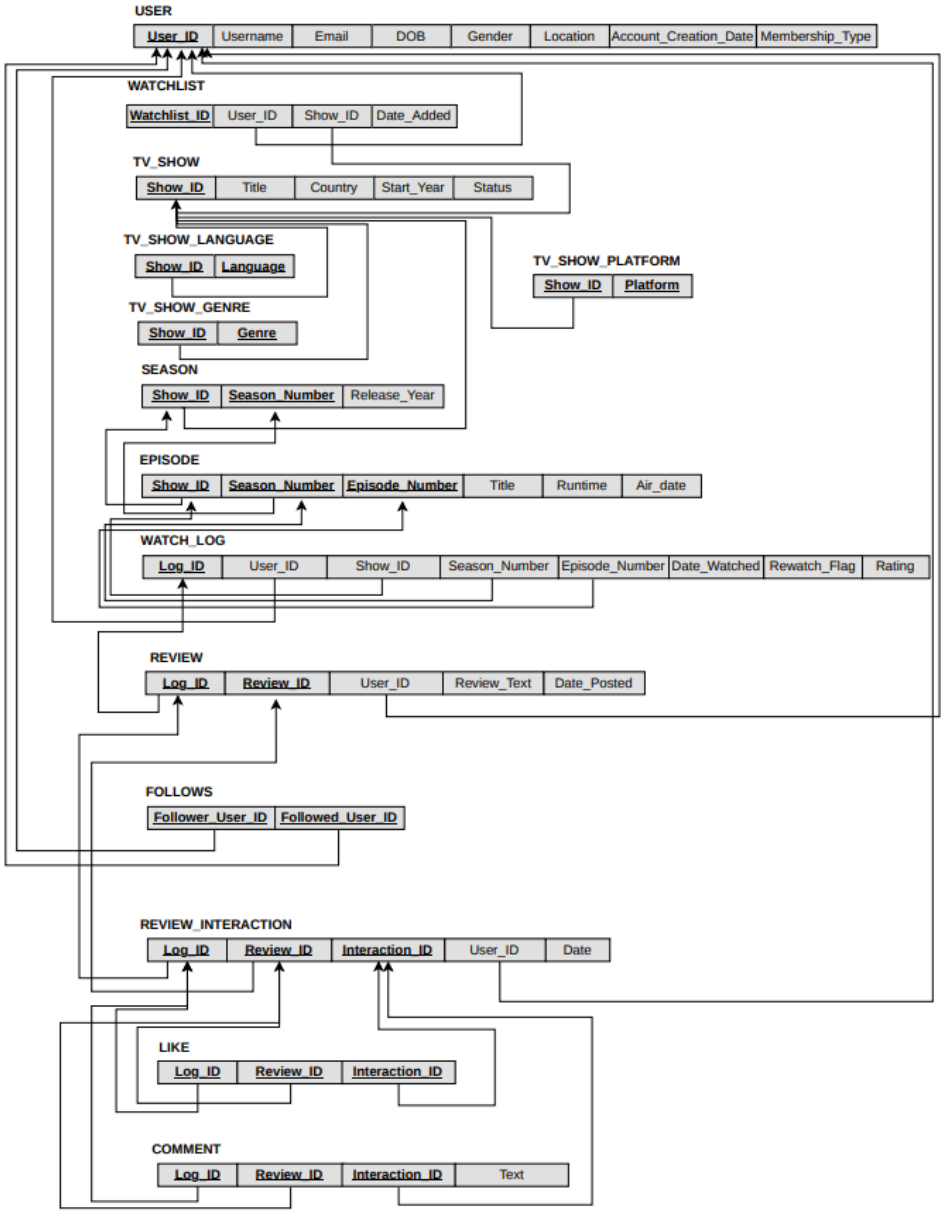
This project will be helpful in…

- Assisting platforms in making informed judgements on popular trends
- Detecting drop of points during show watch times
- Provide user engagement statistics

# EER Diagram


Membership_Type
Username
Email
DOB
User_ID
Account_Creation_Date
Age
Location
Gender
USER
FOLLOWS
(0,N)
(1,M)
FOLLOWED
(0,N)
INTERACTS
(1,1)
Text
Comment
d
Like
REVIEW INTERACTION
Interaction_ID
Date
(1,1)
ADDS TO
(1,1)
Date Added
WATCHLIST
Watchlist ID
(1,1)
LISTED IN
(0,N)
Show ID
Title
Platform
Language
TV Show
Country
Start Year
Status
Genre
(1,N)
HAS
(1,1)
SEASON
Season Number
Release _Year
(1,N)
CONTAINS
(1,1)
EPISODE
Air_date
Runtime
Episode Number
Title
LOGS
(0,N)
(1,1)
Rating (0-5)
Log_ID
Date_ Watched
WATCH_LOG
Rewatch _Flag
(0,1)
IS_LOGGED_IN
(0,N)
WRITES
(1,1)
HAS
(1,1)
RECEIVES
(0,N)
REVIEW
Review ID
Review_Text
Date_ Posted

# Relational Schema



# Project Overview

*Total Tables: 14*
*Total Rows: 1,411*

*USER : 50 rows*
*TV_SHOW : 45 rows*
TV_SHOW_GENRE *: 119 rows*
TV_SHOW_LANGUAGE *: 48 rows*
TV_SHOW_PLATFORM *: 88 rows*
SEASON *: 118 rows*
EPISODE *: 559 rows*
WATCHLIST *: 71 rows*
FOLLOWS *: 48 rows*
WATCH_LOG *: 115 rows*
REVIEW *: 50 rows*
REVIEW_INTERACTION *: 50 rows*
LIKES & COMMENTS: *50 rows*

# Project Overview

Data Sources

- Faker Library – users, reviews, interactions, timestamps
- Random module – ratings, platform assignment, watch behavior
- TVMaze API – real TV show metadata (titles, genres, episodes)

Design –

Used real show information and consistent relationships to maintain authenticity

Constraints –

Limited country distribution, locale mapping to improve realism, restricted platform options, episode/season limits to keep the number of tuples manageable, varied user activity levels to simulate engagement patterns

# Project Goal

Were Business Goals met?

- Overall, most business goals are supported through the analysis of engagement, ratings, demographics, trends and watchlist actions.

# Most Insightful Business Goal

English Business Goal:

*Which TV shows generate the highest levels of user engagement?*

User engagement reflects how actively users interact with content and is a stronger indicator of audience interest than just ratings.

By analyzing engagement, the platform can:

- Identify which shows generate more discussion and activity
- Understand which seasons/episodes garner the most interaction
- Prioritize highly engaging shows for promotion and recommendations
- Can inform marketing strategies based on user behavior

# Project Goal

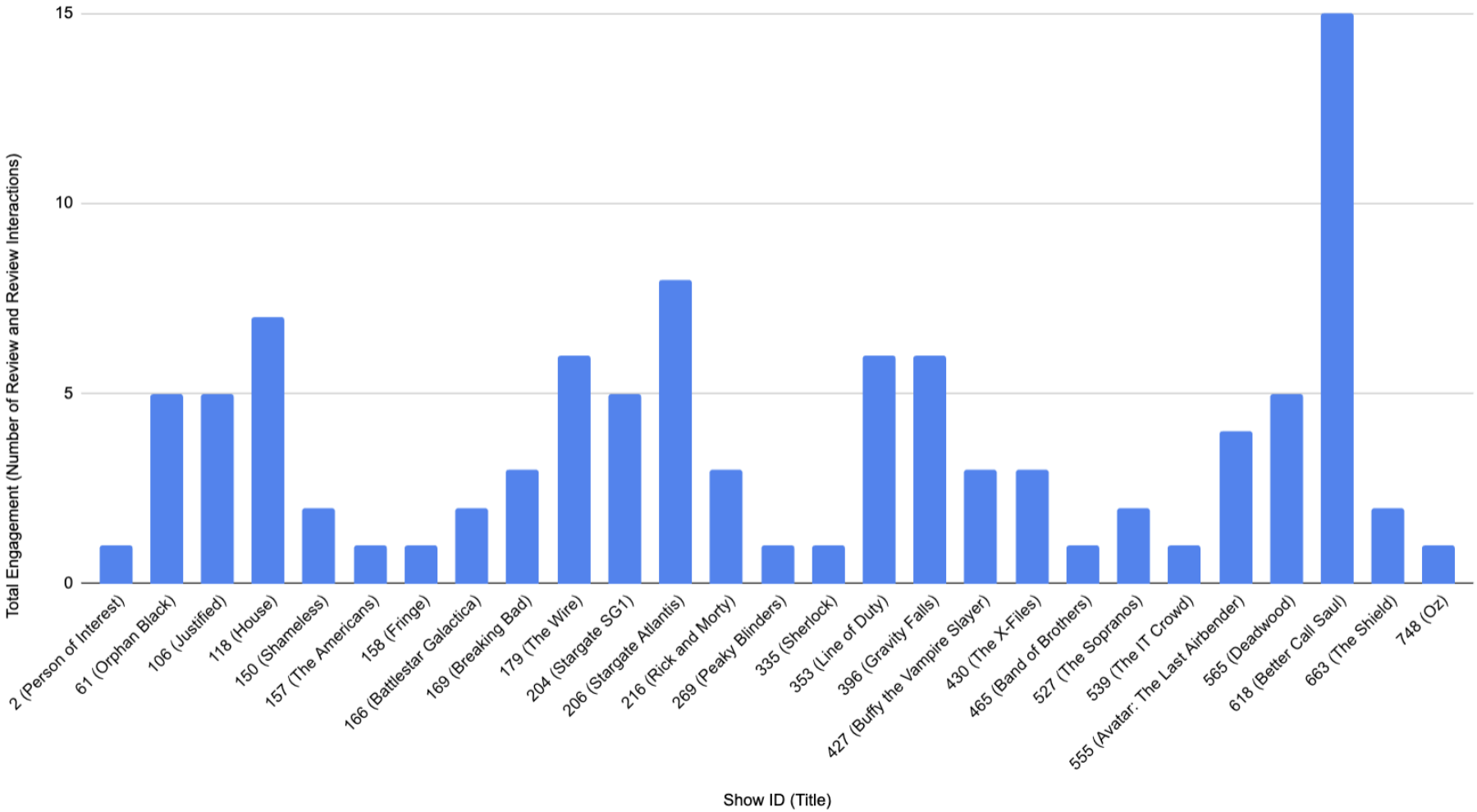
```
SELECT
        ep.Show_ID,
        ep.Season_Number,
         COUNT(DISTINCT r.Review_ID) AS Total_Reviews,
         COUNT(DISTINCT i.Interaction_ID) AS Total_Interactions,
         (COUNT(DISTINCT r.Review_ID) + COUNT(DISTINCT i.Interaction_ID))
AS Total_Engagement
FROM Spring26_S008_T3_EPISODE ep
JOIN Spring26_S008_T3_WATCH_LOG w ON ep.Show_ID = w.Show_ID
         AND ep.Season_Number = w.Season_Number
         AND ep.Episode_Number = w.Episode_Number
JOIN Spring26_S008_T3_REVIEW r ON w.Log_ID = r.Log_ID
LEFT JOIN Spring26_S008_T3_REVIEW_INTERACTION i
       ON r.Review_ID = i.Review_ID
GROUP BY ROLLUP(ep.Show_ID, ep.Season_Number)
ORDER BY Show_ID ASC, Season_Number ASC;
```

# Project Goal

| SHOW_ID | SEASON_NUMBER | TOTAL_REVIEWS | TOTAL_INTERACTIONS | TOTAL_ENGAGEMENT | |
|---|---|---|---|---|---|
| 2 | 3 | 1 | 0 | 1 | |
| 2 | | 1 | 0 | 1 | |
| 61 | 2 | 2 | 3 | 5 | <--- Season Total |
| 61 | | 2 | 3 | 5 | <--- Show Total |
| 106 | 1 | 2 | 3 | 5 | |
| 106 | | 2 | 3 | 5 | |
| 118 | 1 | 3 | 4 | 7 | |
| 118 | | 3 | 4 | 7 | |
| 150 | 1 | 1 | 1 | 2 | |
| 150 | | 1 | 1 | 2 | |
| 157 | 3 | 1 | 0 | 1 | |
| 157 | | 1 | 0 | 1 | |
| 158 | 1 | 1 | 0 | 1 | |
| 158 | | 1 | 0 | 1 | |
| 166 | 3 | 1 | 1 | 2 | |
| 166 | | 1 | 1 | 2 | |
| 169 | 1 | 1 | 0 | 1 | |
| 169 | 2 | 1 | 1 | 2 | |
| 169 | | 2 | 1 | 3 | |
| 179 | 2 | 1 | 1 | 2 | |
| 179 | 3 | 1 | 3 | 4 | |
| 179 | | 2 | 4 | 6 | |
| 204 | 1 | 1 | 1 | 2 | |
| 204 | 3 | 1 | 2 | 3 | |
| 204 | | 2 | 3 | 5 | |
| 206 | 1 | 2 | 2 | 4 | |
| 206 | 2 | 2 | 2 | 4 | |
| 206 | | 4 | 4 | 8 | |
| 216 | 1 | 1 | 0 | 1 | |
| 216 | 2 | 1 | 1 | 2 | |
| 216 | | 2 | 1 | 3 | |
| 269 | 3 | 1 | 0 | 1 | |
| 269 | | 1 | 0 | 1 | |
| 335 | 1 | 1 | 0 | 1 | |
| 335 | | 1 | 0 | 1 | |
| 353 | 1 | 3 | 3 | 6 | |
| 353 | | 3 | 3 | 6 | |
| 396 | 1 | 1 | 1 | 2 | |
| 396 | 2 | 2 | 2 | 4 | |
| 396 | | 3 | 3 | 6 | |
| 427 | 1 | 1 | 0 | 1 | |
| 427 | 2 | 1 | 1 | 2 | |
| 427 | | 2 | 1 | 3 | |
| 430 | 1 | 2 | 1 | 3 | |
| 430 | | 2 | 1 | 3 | |
| 465 | 1 | 1 | 0 | 1 | |
| 465 | | 1 | 0 | 1 | |
| 527 | 2 | 1 | 1 | 2 | |
| 527 | | 1 | 1 | 2 | |
| 539 | 3 | 1 | 0 | 1 | |
| 539 | | 1 | 0 | 1 | |
| 555 | 1 | 1 | 3 | 4 | |
| 555 | | 1 | 3 | 4 | |
| 565 | 1 | 1 | 1 | 2 | |
| 565 | 2 | 1 | 2 | 3 | |
| 565 | | 2 | 3 | 5 | |
| 618 | 1 | 1 | 1 | 2 | |
| 618 | 2 | 3 | 7 | 10 | |
| 618 | 3 | 2 | 1 | 3 | |
| 618 | | 6 | 9 | 15 | |
| 663 | 2 | 1 | 1 | 2 | |
| 663 | | 1 | 1 | 2 | |
| 748 | 1 | 1 | 0 | 1 | |
| 748 | | 1 | 0 | 1 | |
| | | 50 | 50 | 100 | Grand Totals ---> |

65 rows selected.

# Total User Engagement for TV Shows Across All Seasons



# Project Goal: Human and Gen-AI

## The prompt inserted into Gemini

Given the relational schema, generate a SQL query that answers the following question: Which TV shows and seasons generate the highest levels of user engagement? Only include shows or seasons that have at least 1 review or interaction. User engagement is measured by the sum of the number of distinct reviews per episode and number of interactions (likes, comments) on those reviews.

Results are grouped by show and season using ROLLUP to include:
- season-level totals
- show-level totals
- grand total across all shows

add "Spring26_S008_T3_" in front of each table name

## Our SQL Query

```sql
SELECT
    ep.Show_ID,
    ep.Season_Number,
    COUNT(DISTINCT r.Review_ID) AS Total_Reviews,
    COUNT(DISTINCT i.Interaction_ID) AS Total_Interactions,
    (COUNT(DISTINCT r.Review_ID) + COUNT(DISTINCT i.Interaction_ID)) AS Total_Engagement
FROM Spring26_S008_T3_EPISODE ep
JOIN Spring26_S008_T3_WATCH_LOG w ON ep.Show_ID = w.Show_ID
    AND ep.Season_Number = w.Season_Number
    AND ep.Episode_Number = w.Episode_Number
JOIN Spring26_S008_T3_REVIEW r ON w.Log_ID = r.Log_ID
LEFT JOIN Spring26_S008_T3_REVIEW_INTERACTION i ON r.Review_ID = i.Review_ID
GROUP BY ROLLUP(ep.Show_ID, ep.Season_Number)
ORDER BY Show_ID ASC, Season_Number ASC;
```

## AI-Generated SQL Query

```sql
SELECT
    ts.Title AS Show_Title,
    s.Season_Number,
    COUNT(DISTINCT r.Review_ID) + COUNT(ri.Interaction_ID) AS Total_Engagement
FROM
    Spring26_S008_T3_TV_SHOW ts
JOIN
    Spring26_S008_T3_SEASON s ON ts.Show_ID = s.Show_ID
JOIN
    Spring26_S008_T3_EPISODE e ON s.Show_ID = e.Show_ID AND s.Season_Number = e.Season_Number
JOIN
    Spring26_S008_T3_WATCH_LOG wl ON e.Show_ID = wl.Show_ID
        AND e.Season_Number = wl.Season_Number
        AND e.Episode_Number = wl.Episode_Number
JOIN
    Spring26_S008_T3_REVIEW r ON wl.Log_ID = r.Log_ID
LEFT JOIN
    Spring26_S008_T3_REVIEW_INTERACTION ri ON r.Review_ID = ri.Review_ID
GROUP BY
    ROLLUP(ts.Title, s.Season_Number)
HAVING
    COUNT(DISTINCT r.Review_ID) + COUNT(ri.Interaction_ID) >= 1
ORDER BY
    Total_Engagement DESC;
```

- The AI-generated query displays the show's name. Our query only showed Show_ID
- AI-generated query sorts by descending amount of Total Engagement.
  Our query sorts by ascending Show_IDs.
- The AI query does not group seasons by shows. Our query ensures that seasons remain grouped together based on the show.

# Project Goal: Human and Gen-AI

| SHOW_TITLE | SEASON_NUMBER | TOTAL_ENGAGEMENT |
|---|---|---|
| | | 100 |
| Better Call Saul | | 15 |
| Better Call Saul | 2 | 10 |
| Stargate Atlantis | | 8 |
| House | 1 | 7 |
| House | | 7 |
| Line of Duty | 1 | 6 |
| Gravity Falls | | 6 |
| The Wire | | 6 |
| Line of Duty | | 6 |
| Stargate SG??1 | | 5 |
| Orphan Black | | 5 |
| Justified | 1 | 5 |
| Justified | | 5 |
| Orphan Black | 2 | 5 |
| Deadwood | | 5 |
| Avatar: The Last Airbender | 1 | 4 |
| Gravity Falls | 2 | 4 |
| Stargate Atlantis | 1 | 4 |
| Stargate Atlantis | 2 | 4 |
| The Wire | 3 | 4 |
| Avatar: The Last Airbender | | 4 |
| Stargate SG??1 | 3 | 3 |
| Buffy the Vampire Slayer | | 3 |
| The X-Files | 1 | 3 |
| Deadwood | 2 | 3 |
| Better Call Saul | 3 | 3 |
| The X-Files | | 3 |
| Breaking Bad | | 3 |
| Rick and Morty | | 3 |
| Gravity Falls | 1 | 2 |
| Deadwood | 1 | 2 |
| Buffy the Vampire Slayer | 2 | 2 |
| Breaking Bad | 2 | 2 |
| Better Call Saul | 1 | 2 |
| Battlestar Galactica | 3 | 2 |
| The Sopranos | | 2 |
| The Shield | | 2 |
| Shameless | | 2 |
| Battlestar Galactica | | 2 |
| The Wire | 2 | 2 |
| The Sopranos | 2 | 2 |
| The Shield | 2 | 2 |
| Stargate SG??1 | 1 | 2 |
| Shameless | 1 | 2 |
| Rick and Morty | 2 | 2 |
| The IT Crowd | | 1 |
| Peaky Blinders | 3 | 1 |
| Fringe | 1 | 1 |
| Band of Brothers | | 1 |
| The Americans | | 1 |
| Buffy the Vampire Slayer | 1 | 1 |
| Breaking Bad | 1 | 1 |
| Fringe | | 1 |
| Band of Brothers | 1 | 1 |
| The IT Crowd | 3 | 1 |
| The Americans | 3 | 1 |
| Oz | 1 | 1 |
| Person of Interest | | 1 |
| Sherlock | 1 | 1 |
| Peaky Blinders | | 1 |
| Oz | | 1 |
| Rick and Morty | 1 | 1 |
| Person of Interest | 3 | 1 |
| Sherlock | | 1 |

65 rows selected.

# Project Lookback

Key Challenges

- Database Design and Relationships : Early relationship design increased later query complexity
- SQL Queries : Advanced queries (ROLLUP, nested queries, division) required more careful consideration when handling

Improvements

- Clearer Mapping from Business Goals to Queries : make business goals more aligned with SQL programming style
- Refine ER Diagrams and Relationships more before implementing

# Project Takeaway

We built a functional and scalable database system with analytics.

This experience strengthens our resume through :

- database design -> essential for large data sets and ML models
- SQL -> experience in utilizing a new programming language
- Real-world problems -> practical experience with analytics

To expand upon this project, we will …

- Build a front end web app
- Integrate real time streaming data
- Predictive data modeling

# Discussion